

AI-ASSISTED CODING

Name:N.Samuel Raj

Enroll-No:2403A52090

Batch-04

Sub-Group: 'L'

Question-1 (L.1):

Prompt:

You have a function `convert(amount, ccy)` that multiplies an amount by `fetch_rate(ccy)`. Normally, `fetch_rate` uses the network. Rewrite `convert` so it can take a custom rate function.

CODE:

```
ai2 test.py > ...
1  def fetch_rate(ccy):
2      # Normally fetches rate from network
3      # Placeholder implementation
4      raise NotImplementedError("Network fetch not implemented.")
5
6
7  def convert(amount, ccy, rate_func=fetch_rate):
8      return amount * rate_func(ccy)
9
10 # Example: custom rate function for testing
11
12
13 def custom_rate(ccy):
14     rates = {'USD': 150.0}
15     return rates.get(ccy, 1.0)
16
17
18 # Test without network
19 result = convert(10, 'USD', rate_func=custom_rate)
20 rate = custom_rate('USD') # get the rate for display
21 print(f"convert(10,'USD') with rate {rate} => {result}")
```

OUTPUT:

```
s/Desktop/python/ai2 test.py"
convert(10,'USD') with rate 150.0 => 1500.0
```

OBSERVATION:

This code makes the convert function testable without network calls by allowing a custom rate function. It multiplies the amount by the rate provided by that function. The output shows both the rate used and the converted result.

Question-2 (L.2):

Prompt:

Write a recursive Python function that flattens a nested dict into dot-separated keys and uses [index] notation for lists. Return a new dict.

CODE:

```
ai-labtest2.py > ...
1 def flatten(obj, parent_key='', result=None):
2     if result is None:
3         result = {}
4     if isinstance(obj, dict):
5         for k, v in obj.items():
6             new_key = f"{parent_key}.{k}" if parent_key else k
7             flatten(v, new_key, result)
8     elif isinstance(obj, list):
9         for i, v in enumerate(obj):
10            new_key = f"{parent_key}[{i}]"
11            flatten(v, new_key, result)
12    else:
13        result[parent_key] = obj
14    return result
15
16
17 nested = {'user': {'id': 2090, 'name': 'samuel raj'}, 'meta': {'lang': 'en'}}
18 flat = flatten(nested)
19 print(flat)
```

OUTPUT:

```
/Desktop/python/ai-labtest2.py
{'user.id': 2090, 'user.name': 'samuel raj', 'meta.lang': 'en'}
```

OBSERVATION:

This code takes a nested dictionary or list and turns it into a single-level dictionary with keys joined by dots and list indexes in brackets